



OpenViking 文档入库处理流程说明

从资源提交到知识树、摘要、向量索引和状态完成的全链路说明

项目	内容
文档目的	让读者通过流程、输入输出和源码映射，理解 OpenViking 如何完成文档入库处理。
覆盖范围	资源接入、来源校验、解析器选择、临时目录、URI 落位、正式落盘、异步摘要、向量化、状态更新。
不覆盖范围	会话记忆处理、检索召回策略、上层问答生成逻辑；这些属于其他流程文档。
版本口径	基于当前工作区源码梳理，生成日期 2026-04-27。
核心源码	resources.py、resource_service.py、resource_processor.py、media_processor.py、registry.py、tree_builder.py、summarizer.py、semantic_processor.py、semantic_dag.py

一句话说明

文档入库不是简单上传文件，而是一个“接收资源 -> 解析成临时知识树 -> 解析最终 URI -> 原子落盘/增量同步 -> 异步生成 L0/L1 -> 写入向量索引 -> 更新处理状态”的流水线。

1. 总体处理链路

文档入库的目标，是把外部文件、目录、网页、代码仓库或云文档转换为 OpenViking 内部的知识节点。入库完成后，系统不仅保存原文或转换后的 L2 内容，还会生成用于检索的 .abstract.md、.overview.md，并把这些信息写入向量索引。

入口

HTTP POST /api/v1/resources
或 SDK/Local Client add_resource()

同步阶段

1. 资源参数校验和身份上下文解析
2. ResourceService 派生目标 URI、注册知识文档回调
3. ResourceProcessor 调用 UnifiedResourceProcessor
4. ParserRegistry 选择解析器，Parser 写入临时 VikingFS
5. TreeBuilder 解析最终 root_uri 和 temp_uri
6. ResourceProcessor 将 temp 文档树移动到正式 AGFS
7. Summarizer 投递 SemanticMsg

异步阶段

8. SemanticProcessor / SemanticDag 自底向上生成 L0/L1
9. Embedding / Vectorize 写入向量索引
10. KnowledgeDocumentRegistry 更新 ready 或 failed 状态

阶段	同步/异步	主要输入	主要输出
接入校验	同步	path/temp_file_id、to、parent、folder_path、headers	合法 source、RequestContext、处理参数
解析切分	同步	source、instruction、strict/include/exclude	ParseResult、temp_dir_path、source_format、warnings
URI 落位	同步	temp_dir_path、scope、to/parent、source_path	root_uri、temp_uri、candidate_uri
正式落盘	同步	temp_uri、root_uri、锁	正式 VikingFS/AGFS 文档树
任务投递	同步	root_uri、temp_uri、document_id、ctx	SemanticMsg 入队，processing_enqueued=true
摘要索引	异步	SemanticMsg、目录树、原文内容	.abstract.md、.overview.md、向量记录
状态完成	异步/同步等待	队列状态、embedding 结果	knowledge document ready/failed、queue_status

阅读方式

后续章节按真实执行顺序展开。每一步都说明入口、执行模块、输入、输出、写入位置和异常处理，读者可以沿着 root_uri 和 document_id 跟完整条链路。

2. 步骤一：资源提交与接入校验

文档入库可以从 HTTP、SDK 或本地嵌入式 Client 发起。HTTP 模式下，前端或业务系统通常先调用 `temp_upload` 上传文件，再把 `temp_file_id` 交给 `/api/v1/resources`；远程 URL、代码仓库 URL、飞书链接等也可以直接作为 `path` 提交。

入口	关键参数	说明
POST <code>/api/v1/resources/temp_upload</code>	<code>file</code>	保存上传文件，返回 <code>temp_file_id</code> 和 <code>original_filename</code> 。
POST <code>/api/v1/resources</code>	<code>path</code> 或 <code>temp_file_id</code>	提交资源处理任务。二者必须至少提供一个。
SDK <code>add_resource</code>	<code>path</code> 、 <code>to</code> 、 <code>parent</code> 、 <code>wait</code> 、 <code>timeout</code> 等	本地或 HTTP Client 封装入库调用。

参数	作用	处理逻辑
<code>path</code>	远程资源地址或本地路径	HTTP 直连模式要求远程来源；SDK/本地模式可允许本地路径。
<code>temp_file_id</code>	临时上传文件编号	由 <code>resolve_uploaded_temp_file_id</code> 解析为上传目录下的真实文件。
<code>to</code>	精确目标 URI	例如 <code>viking://resources/product/manual</code> ；不能与 <code>parent</code> 同时使用。
<code>parent</code>	目标父目录	<code>TreeBuilder</code> 在该父目录下根据文档名生成最终 URI。
<code>folder_path</code>	知识管理 UI 的虚拟文件夹	不直接改变内部资源树，可用于知识文档记录归类。
<code>instruction/reason</code>	处理提示	会作为语义摘要阶段的提示信息，提高 L0/L1 与场景相关性。
<code>wait/timeout</code>	是否等待异步处理完成	<code>wait=true</code> 时等待队列完成；超时抛 <code>DeadlineExceededError</code> 。
<code>include/exclude/ignore_dirs/strict</code>	目录或仓库解析策略	透传给 <code>DirectoryParser</code> 和扫描逻辑。
<code>watch_interval</code>	资源监控周期	大于 0 时创建或更新 <code>watch task</code> ，用于后续自动同步。

安全边界

HTTP Server 不接受任意主机本地路径作为 `path`，除非文件先经过 `temp_upload`。这个设计避免外部调用方通过入库接口读取服务端文件系统。

3. 步骤二：ResourceService 编排入库请求

`ResourceService.add_resource` 是资源入库的业务编排层。它不直接解析文件，而是负责参数治理、目标 URI 约束、知识文档记录回调、等待队列和 `watch task` 管理。

```
ResourceService.add_resource()
```

```
-> _ensure_initialized()
-> register_wait_telemetry(wait)
-> derive effective_to when folder_path exists
-> validate to/parent scope == resources
-> create post_finalize_hook
-> ResourceProcessor.process_resource(...)
-> attach document_id / knowledge_document
-> wait_complete if wait=true
-> create/cancel watch task if needed
```

处理点	输入	输出/副作用
目标 URI 检查	to、parent	只允许 resources scope，防止资源接口写入 user/agent/session 空间。
默认 URI 派生	path、source_ref、folder_path	当未指定 to/parent 且存在 folder_path 时，生成 viking://resources 下的默认路径。
知识文档回调	finalize_result	在 TreeBuilder 确定 root_uri 后注册 knowledge document，得到 document_id。
状态初始化	processing_requested	若后续需要语义处理，状态置为 processing；否则置为 ready。
等待队列	wait、timeout	等待 QueueManager.wait_complete，返回 queue_status 或超时。
资源监控	watch_interval、effective_to	创建、更新或取消 watch task。

关键结论

ResourceService 是“业务控制面”，ResourceProcessor 是“处理执行面”。前者决定这次入库应该如何被登记、等待和监控，后者负责真的把资料变成知识树。

4. 步骤三：来源路由、解析器选择与临时知识树

ResourceProcessor 首先调用 UnifiedResourceProcessor.process。该模块根据 source 的形态决定如何处理：URL 走远程解析，本地目录走 DirectoryParser，本地文件走文件解析，ZIP 先安全解压再按目录处理，原始文本则直接交给 parse。

```
UnifiedResourceProcessor.process(source)
if source is URL:
    Feishu URL -> FeishuParser
    Git/repo URL -> CodeRepositoryParser
    other URL -> HTMLParser
elif source is existing local directory:
    DirectoryParser.parse()
elif source is existing local file:
    .zip -> safe_extract_zip -> DirectoryParser
    other -> ParserRegistry.parse(file_path)
elif source looks like forbidden local path:
    PermissionDeniedError
else:
    treat as raw content -> parse(source)
```

解析器	典型输入	输出重点
Markdown/Text	.md、.txt、原始文本	按标题或文本块切分，写入 Markdown/Text 节点。
PDF	.pdf	提取页面文本、表格和图片信息，必要时结合 VLM 处理非文本内容。
Word/Excel/PowerPoint	.docx/.doc、.xlsx、.pptx	转换办公文档内容，保留表格、章节和页面语义。
HTML/Feishu	网页 URL、飞书文档 URL	抓取或导出在线文档，再转为内部临时目录。
CodeRepository	GitHub/GitLab/git/ssh URL	拉取仓库，按代码/文档/配置文件组织目录树。
Directory/ZIP	本地目录、压缩包	递归扫描目录，按 include/exclude/ignore_dirs 过滤。
Image/Audio/Video	图片、音频、视频	生成媒体摘要，作为可检索文本进入知识树。

Parser 的重要原则是：只做格式转换、结构拆分和临时文件写入，不负责最终落盘，也不直接完成全局向量索引。解析输出统一封装为 ParseResult。

ParseResult 字段	含义	后续使用方
temp_dir_path	Parser 写入的临时 VikingFS 目录	TreeBuilder 和 ResourceProcessor
source_path	真实来源路径或 URL	TreeBuilder、知识文档记录、审计
source_format	来源类型，例如 pdf、word、repository	TreeBuilder、Summarizer、SemanticDag
meta	解析器产生的元数据	ResourceProcessor 返回结果和后续观察
warnings	解析中的非致命警告	ResourceService 返回 errors/warnings

5. 步骤四：TreeBuilder 解析最终 URI

Parser 会在临时 VikingFS 下创建一个文档根目录。TreeBuilder.finalize_from_temp 读取这个临时目录，要求其中只有一个有效文档根目录，然后计算正式 root_uri。这个阶段只确定结构和 URI，不直接生成摘要。

```
TreeBuilder.finalize_from_temp(temp_dir_path, ctx, scope, to_uri, parent_uri)
-> ls(temp_uri)，找到唯一文档根目录
-> sanitize document name
-> 如果 source_format == repository，Git URL 可转为 org/repo
-> base_uri = parent_uri 或 viking://resources
-> candidate_uri = to_uri 或 base_uri / final_doc_name
-> 若未指定 to_uri，则自动 resolve_unique_uri
-> 返回 BuildingTree(root.uri=root_uri, root.temp_uri=temp_doc_uri)
```

场景	root_uri 规则
指定 to	使用 to 作为精确目标 URI，要求资源接口只写 resources scope。
指定 parent	检查 parent 存在且为目录，在 parent 下拼接文档名。
未指定 to/parent	默认落到 viking://resources/{文档名}。
名称冲突	自动追加 _1、_2 ... 直到找到可用 URI。
代码仓库	若可解析 org/repo，则最终路径用仓库组织名和仓库名表达。
媒体资源	根据媒体类型选择相应资源基路径。

6. 步骤五：正式落盘、生命周期锁与增量更新

TreeBuilder 返回 root_uri 与 temp_uri 后，ResourceProcessor 负责把临时文档树移动到正式 AGFS。这里分首次添加和增量更新两种路径。

路径	判断条件	处理方式
首次添加	root_uri 不存在	创建父目录，在 point lock 保护下把 temp_uri 对应目录 mv 到 root_uri。
增量更新	root_uri 已存在	不立即覆盖目标目录，而是为目标目录获取 SUBTREE lifecycle lock，后续由 SemanticDag 做 diff 同步。
锁失败	无法获取 lifecycle lock	优雅降级为空 handle，但会记录 warning；后续处理仍尽量继续。
finalize 失败	TreeBuilder 或移动前出错	删除临时目录，返回 error，避免污染正式知识库。

为什么要先写临时目录

临时目录让解析阶段和正式知识库解耦：解析失败不会留下半成品；首次添加可以用移动操作快速落盘；增量

更新可以先生成新版本临时树，再由后续 DAG 对比差异。

7. 步骤六：注册知识文档与投递语义任务

正式 root_uri 已经确定后，ResourceService 的 post_finalize_hook 会注册知识文档记录。随后 ResourceProcessor 根据 build_index 和 summarize 判断是否需要语义处理。默认 build_index=true，因此一般都会投递 SemanticMsg。

```
ResourceProcessor after move/finalize
should_summarize = summarize or build_index
processing_requested = should_summarize

post_finalize_hook(result):
    register knowledge document
    processing_status = processing if should_summarize else ready
    result.document_id = document_id

Summarizer.summarize(
    resource_uris=[root_uri],
    temp_uris=[temp_uri_for_summarize],
    skip_vectorization=not build_index,
    lifecycle_lock_handle_id=handle_id,
    document_id=document_id
)

SemanticQueue.enqueue(SemanticMsg)
```

SemanticMsg 字段	值来源	作用
uri	temp_uri 或 root_uri	SemanticProcessor 实际读取和处理的目录。
target_uri	root_uri, 当 uri 与 root_uri 不同	增量/同步时用于把临时树同步到正式树。
context_type	由 root_uri 推导，一般为 resource	决定向量记录和上下文类型。
account_id/user_id/agent_id/role	RequestContext	用于权限、owner_space 和索引过滤。
skip_vectorization	not build_index	允许只生成摘要，不写向量索引。
lifecycle_lock_handle_id	ResourceProcessor 获取的 SUBTREE 锁	保护语义处理和增量同步期间的目录一致性。
is_code_repo	source_format == repository	让 SemanticDag 按代码仓库策略处理。
document_id	KnowledgeDocumentRegistry	处理完成后更新 ready/failed 状态。

8. 步骤七：SemanticProcessor 生成 L0/L1 并向量化

SemanticProcessor 从 SemanticQueue 取出 SemanticMsg。如果是 resource 类型，它会创建 SemanticDagExecutor，从根目录开始调度目录节点。DAG 的核心策略是自底向上：先处理文件和子目录，再生成父目录 overview/abstract。

```
SemanticProcessor.on_dequeue(msg)
-> ctx_from_semantic_msg(msg)
-> if msg.target_uri exists and msg.uri != msg.target_uri:
    incremental_update = true
```

```
-> SemanticDagExecutor.run(msg.uri)
    dispatch directories
    generate file summaries
    collect child abstracts
    generate .overview.md
    extract .abstract.md
    write summary files
    schedule vectorize_file / vectorize_directory
-> EmbeddingTaskTracker waits vector tasks
-> update document status ready/failed
-> release lifecycle lock
```

产物	写入位置	用途
L2 内容	root_uri 下的正文、章节、转换文本或媒体摘要文件	最终回答引用、证据展开、精确读取。
.overview.md	每个目录节点下	目录级概览，用于层级导航、重排和上下文理解。
.abstract.md	每个目录节点下	短摘要，用于快速召回和过滤。
文件向量	VectorDB	让叶子文件或章节可被语义检索命中。
目录向量	VectorDB	让 L0/L1 目录节点可被先召回，再向下展开。
document status	KnowledgeDocumentRegistry	ready 或 failed，供管理 UI 和业务调用查看。

增量更新如何生效

当 target_uri 已存在且 msg.uri 是临时新树时，SemanticDag 会在完成摘要和向量任务后执行 sync diff，把新增、修改、删除同步到正式目录，并移动或清理对应 L0/L1 索引。

9. wait、状态流转与异常处理

入库接口可以异步返回，也可以 `wait=true` 等待队列处理完成。无论是否等待，系统都会围绕 `result`、`document_id`、`queue_status` 和 `processing_status` 提供可观测状态。

状态/字段	何时产生	含义
<code>result.status=success</code>	解析和投递阶段成功	表示同步阶段完成，不一定代表摘要和索引已全部完成。
<code>processing_requested=true</code>	<code>summarize</code> 或 <code>build_index</code> 为 <code>true</code>	后续需要 <code>SemanticQueue</code> 处理。
<code>processing_enqueued=true</code>	<code>Summarizer</code> 成功 <code>enqueue</code>	语义处理任务已进入队列。
<code>knowledge_document.processing</code>	文档注册后且需要语义处理	管理端可显示处理中。
<code>knowledge_document.ready</code>	队列和向量任务完成	文档可稳定检索。
<code>knowledge_document.failed</code>	语义处理或 <code>embedding</code> 失败	记录 <code>processing_error</code> ，便于重试或排查。
<code>queue_status</code>	<code>wait=true</code> 且 <code>wait_complete</code> 返回	包含队列完成情况，用于同步调用方判断。

异常点	典型原因	系统处理
参数校验失败	<code>path/temp_file_id</code> 缺失， <code>to</code> 与 <code>parent</code> 同时出现，非 <code>resources scope</code>	抛出 <code>InvalidArgumentError</code> 或请求校验错误。
来源权限失败	<code>HTTP path</code> 看起来像服务端本地路径	抛出 <code>PermissionDeniedError</code> ，要求 <code>temp_upload</code> 或远程 <code>URL</code> 。
解析失败	文件不存在、格式损坏、解析器异常	<code>result.status=error</code> ， <code>errors</code> 包含 <code>Parse error</code> 。
无临时内容	<code>Parser</code> 未生成 <code>temp_dir_path</code>	返回 <code>error</code> ，不进入 <code>TreeBuilder</code> 。
落盘失败	<code>URI</code> 冲突、父目录非法、 <code>AGFS</code> 移动异常	清理临时目录，返回 <code>Finalize from temp error</code> 。
语义任务投递失败	<code>QueueManager</code> 或 <code>Summarizer</code> 异常	<code>processing_enqueued=false</code> ，文档状态更新为 <code>failed</code> 。
模型临时错误	<code>LLM/VLM</code> 服务限流或网络问题	<code>SemanticProcessor</code> <code>re-enqueue</code> ，并通过熔断避免重入风暴。
模型永久错误	鉴权、配置或不可恢复错误	记录 <code>failed</code> ，更新 <code>processing_error</code> 。
等待超时	<code>wait=true</code> 且 <code>timeout</code> 到期	抛 <code>DeadlineExceededError</code> ，但后台任务可能仍继续处理。

实现口径

同步返回 `success` 代表“资源已被接收、解析、落盘并成功投递后续处理”，不等价于“向量索引已完成”。若业务要求上传后立即可搜，应使用 `wait=true` 或轮询 `knowledge document` 状态。

10. 示例：上传一个 Word 产品手册

假设用户上传文件“西软产品手册.docx”，前端先调用 temp_upload 得到 temp_file_id，然后提交 add_resource，并指定 folder_path=产品资料、wait=true。

```
1. POST /api/v1/resources/temp_upload
  input: file=西软产品手册.docx
  output: temp_file_id=upload_xxx, original_filename=西软产品手册.docx

2. POST /api/v1/resources
  input:
    temp_file_id=upload_xxx
    folder_path=产品资料
    instruction=面向客服和投标场景提取产品能力、模块、限制和案例
    wait=true

3. ResourceService
  source_ref=西软产品手册.docx
  effective_to=viking://resources/产品资料/西软产品手册

4. WordParser
  temp_dir_path=viking://.tmp/.../西软产品手册
  source_format=word

5. TreeBuilder / ResourceProcessor
  root_uri=viking://resources/产品资料/西软产品手册
  move temp tree -> root_uri
  document_id=...

6. SemanticProcessor
  write:
    viking://resources/产品资料/西软产品手册/.abstract.md
    viking://resources/产品资料/西软产品手册/.overview.md
    viking://resources/产品资料/西软产品手册/{章节}.md
  vectorize:
    directory L0/L1 records
    file/chapter records

7. Final
  knowledge_document.processing_status=ready
  search/find 可以命中该手册内容
```

读者要跟踪的线索	在哪里出现	用途
temp_file_id	temp_upload 返回值	定位上传临时文件。
source_ref	ResourceService.add_resource	保留原始文件名或 URL 作为登记依据。
root_uri	TreeBuilder / ResourceProcessor result	正式知识树入口，也是后续 read/search 的 target_uri。
temp_uri	ParseResult / BuildingTree	语义处理或增量同步期间的临时树。
document_id	KnowledgeDocumentRegistry	查询知识文档处理状态。
SemanticMsg.id	SemanticQueue	跟踪语义处理和 embedding 任务。

telemetry_id	run_operation / telemetry	关联 parse、finalize、wait、queue 等指标。
--------------	---------------------------	-----------------------------------

11. 源码映射

流程步骤	关键源码	职责
HTTP 接入	openviking/server/routers/resources.py	temp_upload、add_resource 路由、请求模型和本地路径保护。
业务编排	openviking/service/resource_service.py	参数校验、document 注册、wait、watch task。
处理执行	openviking/utils/resource_processor.py	解析、TreeBuilder、正式落盘、锁、Summarizer 投递。
来源路由	openviking/utils/media_processor.py	URL、目录、文件、ZIP、原始内容的处理策略。
解析器注册	openviking/parse/registry.py	根据扩展名和 URL 类型选择 parser。
解析器实现	openviking/parse/parsers/*.py	PDF、Word、Excel、PowerPoint、HTML、代码仓库、媒体等解析。
URI 落位	openviking/parse/tree_builder.py	从临时目录解析 root_uri、candidate_uri 和 temp_uri。
任务投递	openviking/utils/summarizer.py	构造 SemanticMsg 并投递 SemanticQueue。
语义消费	openviking/storage/queuefs/semantic_processor.py	消费 SemanticMsg，处理熔断、重试和资源语义任务。
DAG 摘要	openviking/storage/queuefs/semantic_dag.py	自底向上生成 .overview.md/.abstract.md，调度向量任务，更新状态。
向量化	openviking/utils/embedding_utils.py	vectorize_file、vectorize_directory_meta。
状态登记	openviking/service/knowledge_document_registry.py	processing/ready/failed 状态和处理错误。

至此，读者可以从一次 add_resource 请求开始，沿着 source、root_uri、document_id 和 SemanticMsg.id，完整追踪文档从外部资料变成可检索知识库节点的全过程。